

Colorisation automatique d'images

Comment un réseau de neurones apprend à colorier des photos en noir et blanc

Guide Technique – Projet L3 Informatique

Groupe I



Table des matières

1	Introduction	2
2	L'espace colorimétrique Lab	2
2.1	Pourquoi ne pas travailler en RGB ?	2
2.2	Le format CIE Lab	2
3	Le jeu de données : STL-10	3
3.1	Présentation	3
3.2	Préparation des données	3
3.3	Augmentation des données	3
4	L'architecture du réseau : U-Net	4
4.1	Vue d'ensemble	4
4.2	L'encodeur : extraire l'information	4
4.3	Le goulot d'étranglement (bottleneck)	5
4.4	Le mécanisme d'attention	5
4.5	Le décodeur : reconstruire l'image	6
4.6	Les connexions résiduelles (skip connections)	6
4.7	La sortie	6
5	La fonction de perte	7
5.1	Rôle d'une fonction de perte	7
5.2	Une perte personnalisée	7
6	L'entraînement	8
6.1	Paramètres d'entraînement	8
6.2	Le taux d'apprentissage avec Cosine Decay	8
6.3	Callbacks	8
6.4	Courbes d'apprentissage	8
7	Coloriser une image donnée	9
8	Utilisation du programme	10

1 Introduction

La colorisation automatique d'images est le problème suivant : **étant donné une image en niveaux de gris, retrouver une version colorée plausible de cette image**. C'est une tâche que les humains accomplissent naturellement en s'appuyant sur leur connaissance du monde (le ciel est bleu, l'herbe est verte...).

Pourquoi c'est difficile ?

Une pomme peut être de plusieurs couleurs : rouge, verte ou jaune, un manteau peut être de n'importe quelle couleur. Le réseau doit donc apprendre à produire des colorisations **cohérentes et statistiquement plausibles**, qui ne seront pas exactes.

2 L'espace colorimétrique Lab

2.1 Pourquoi ne pas travailler en RGB ?

Les images numériques sont habituellement représentées en **RGB** : chaque pixel est décrit par trois valeurs (rouge, vert, bleu). Ce format est pratique pour l'affichage, mais peu adapté à la colorisation automatique. En RGB, la luminosité et la couleur sont *mélangées* dans les trois canaux, ce qui rend la tâche d'apprentissage plus complexe.

2.2 Le format CIE Lab

L'espace colorimétrique **CIE Lab** (ou $L^*a^*b^*$) représente une image différemment :

- Le canal **L** (Luminance) encode uniquement la luminosité, de 0 (noir) à 100 (blanc). C'est exactement ce que contient une image en niveaux de gris.
- Le canal **a** encode l'axe vert-magenta.
- Le canal **b** encode l'axe bleu-jaune.

L'idée clé

En Lab, coloriser une image revient à **prédire les canaux a et b à partir du canal L** . C'est une séparation nette entre ce qu'on connaît (la luminosité) et ce qu'on veut prédire (la couleur).

3 Le jeu de données : STL-10

3.1 Présentation

Le modèle est entraîné sur le dataset **STL-10**, un jeu de données public contenant 100 000 images non étiquetées de scènes variées (animaux, véhicules, paysages...). Ces images sont en couleur ; elles servent à la fois d'entrée (on garde le canal L) et de cible (on apprend à prédire *ab*).

3.2 Préparation des données

Chaque image suit le pipeline suivant :

1. Redimensionnement à 128×128 pixels.
2. Conversion RGB $\rightarrow$ Lab.
3. Normalisation des canaux comme décrit ci-dessus.
4. Séparation en 90 000 images d'entraînement et 10 000 images de validation.

3.3 Augmentation des données

Pour rendre le modèle plus robuste, des transformations aléatoires sont appliquées pendant l'entraînement :

- **Retournement horizontal.**
- **Variation de luminosité** aléatoire ($\pm 15\%$).
- **Bruit gaussien** léger ajouté aux canaux.

Ces augmentations permettent d'obtenir artificiellement plus de diversité, sans avoir à collecter davantage d'images.

4 L'architecture du réseau : U-Net

4.1 Vue d'ensemble

Le modèle est un **réseau de neurones convolutif** de type **U-Net**. Cette architecture, initialement conçue pour la segmentation d'images médicales, est très adaptée aux tâches où l'on souhaite produire une image à partir d'une autre image de même dimension.

Qu'est-ce qu'un réseau convolutif ?

Un réseau convolutif (CNN) est une architecture spécialisée dans le traitement des images. Il utilise des *filtres* (petites matrices) qui balayent l'image pour en extraire des caractéristiques locales : contours, textures, formes, puis des structures de plus en plus abstraites (objets, scènes).

4.2 L'encodeur : extraire l'information

La première moitié du U-Net est un **encodeur**. Il compresse progressivement l'image en une représentation de plus en plus abstraite et de plus petite résolution spatiale. Le réseau utilise 4 niveaux de compression, avec respectivement 64, 128, 256 et 512 filtres.

À chaque niveau, un **bloc convolutif** est appliqué, composé de :

1. Deux convolutions 3×3 (extraction de caractéristiques locales).
2. Une normalisation par batch (BatchNorm) pour stabiliser l'apprentissage.
3. Une activation ReLU (introduit la non-linéarité).
4. Un dropout (désactive aléatoirement 20 % des neurones pour éviter le surapprentissage).

Après chaque bloc, un **Max Pooling** 2×2 divise par 2 la résolution spatiale.

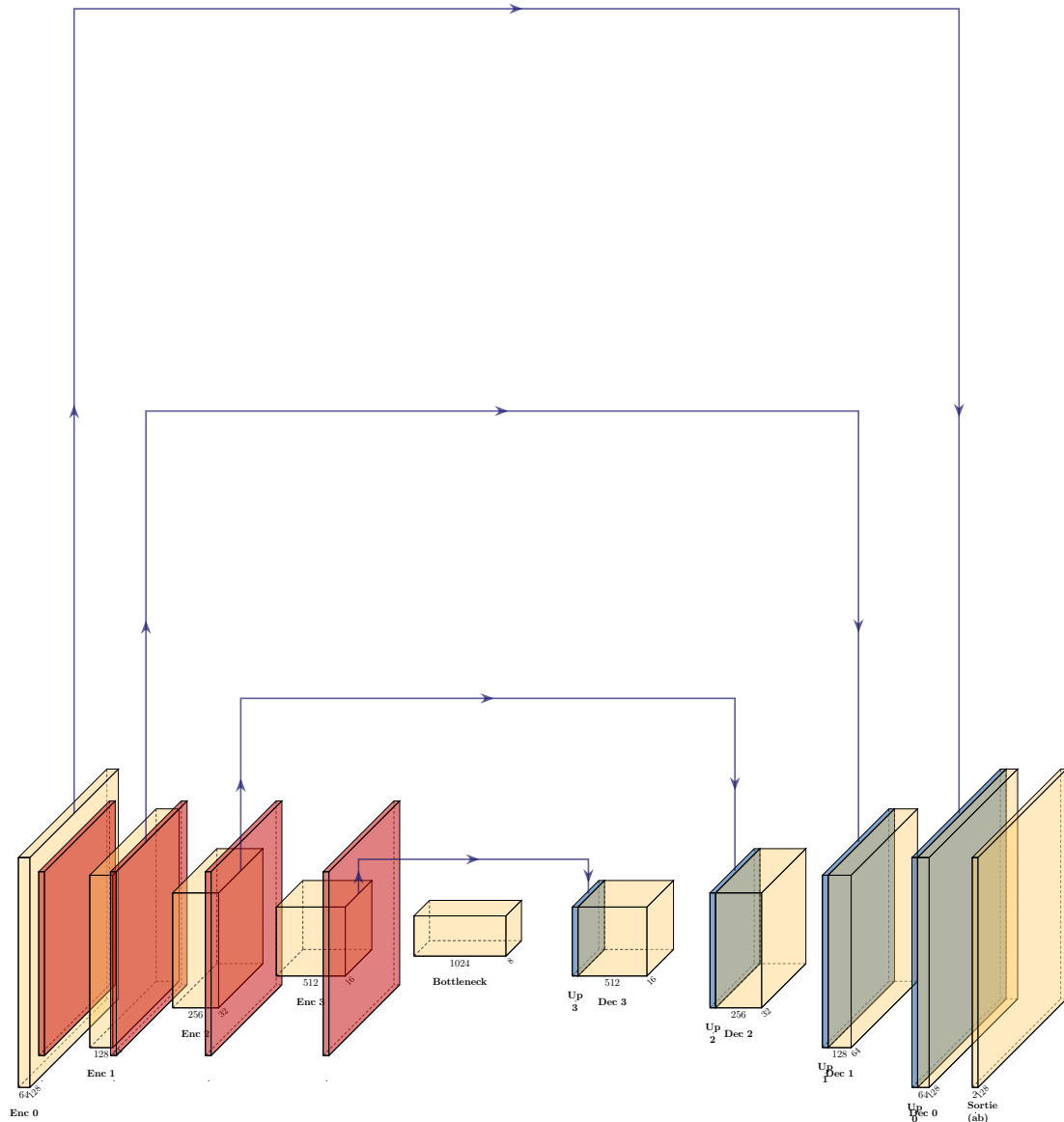


FIGURE 1 – Architecture U-Net : encodeur, goulot d'étranglement, décodeur et connexions résiduelles (skip connections).

4.3 Le goulot d'étranglement (bottleneck)

Au centre du réseau se trouve le **goulot d'étranglement** : la représentation la plus compressée de l'image (8×8 pixels, 1024 filtres). C'est ici que le réseau dispose d'une vue globale de la scène. Un bloc d'attention (voir section suivante) est appliqué à ce niveau.

4.4 Le mécanisme d'attention

Le **bloc d'attention** permet au réseau de pondérer l'importance relative de chaque filtre au goulot d'étranglement. Concrètement :

1. Un GlobalAveragePooling résume chaque carte de caractéristiques en un seul scalaire.
2. Deux couches denses calculent un vecteur de poids entre 0 et 1 (via une sigmoïde).
3. Ces poids multiplient les cartes correspondantes : les filtres jugés importants sont amplifiés, les autres atténués.

Analogie

L'attention, c'est comme mettre en surbrillance les parties d'un texte avant de le résumer. Le réseau apprend à ne retenir que l'essentiel pour décider des couleurs.

4.5 Le décodeur : reconstruire l'image

La seconde moitié du U-Net est le **décodeur**. Il reconstruit progressivement une image de pleine résolution à partir de la représentation compressée. À chaque niveau, une **convolution transposée** (aussi appelée déconvolution) double la résolution spatiale.

4.6 Les connexions résiduelles (skip connections)

La caractéristique la plus distinctive du U-Net est l'existence de **connexions directes** entre chaque niveau de l'encodeur et le niveau symétrique du décodeur. Ces connexions résiduelles (ou *skip connections*) transmettent les détails fins (contours, textures) directement depuis l'encodeur, sans les faire transiter par le goulot d'étranglement.

Pourquoi c'est crucial ?

Sans ces connexions, le décodeur doit reconstruire tous les détails à partir d'une représentation très compressée. Avec elles, les informations de localisation sont préservées, ce qui est indispensable pour produire des couleurs précisément positionnées dans l'image.

4.7 La sortie

La dernière couche est une convolution 1×1 avec activation **tanh**, qui produit une image à 2 canaux (a_{norm} et b_{norm}) dans l'intervalle $[-1, 1]$.

5 La fonction de perte

5.1 Rôle d'une fonction de perte

Lors de l'entraînement, le réseau compare ses prédictions aux vraies valeurs de couleur et calcule une **erreur** (la perte). L'objectif est de minimiser cette erreur en ajustant les poids du réseau.

5.2 Une perte personnalisée

La fonction de perte utilisée par le modèle est personnalisée et combine trois termes :

$$\mathcal{L} = \underbrace{\text{MAE}}_{\text{erreur de base}} + 0,05 \cdot \underbrace{\text{Pénalité de désaturation}}_{\text{évite le gris}} - 0,05 \cdot \underbrace{\text{Variance}}_{\text{encourage la diversité}}$$

Terme 1 – MAE (Mean Absolute Error) C'est la distance absolue moyenne entre les couleurs prédites et les vraies couleurs :

$$\text{MAE} = \frac{1}{N} \sum_i |a_{\text{prédit},i} - a_{\text{réel},i}| + |b_{\text{prédit},i} - b_{\text{réel},i}|$$

Terme 2 – Pénalité de désaturation Sans contrainte particulière, les réseaux ont tendance à prédire des couleurs proches du gris (valeurs a et b proches de 0), car c'est la solution qui minimise le risque d'erreur. Ce terme pénalise explicitement les prédictions trop ternes :

$$\text{Désaturation} = \mathbb{E} \left[\exp \left(-20 \cdot \hat{c}^2 \right) \right]$$

Plus c'est proche de zéro (gris), plus ce terme est élevé et augmente la perte.

Terme 3 – Variance Ce terme récompense la **diversité des couleurs** au sein d'une image. En le soustrayant à la perte, on encourage le réseau à produire des images variées et colorées plutôt qu'uniformes.

En résumé

La fonction de perte fait un compromis : être précis (MAE), ne pas produire d'images ternes (pénalité de désaturation), et produire des images aux couleurs variées (variance).

6 L'entraînement

6.1 Paramètres d'entraînement

Paramètre	Valeur
Taille des images	128×128 pixels
Taille des batchs	128 images
Nombre d'époques max	30
Taux d'apprentissage initial	3×10^{-4}
Optimiseur	Adam

6.2 Le taux d'apprentissage avec Cosine Decay

Le **taux d'apprentissage** contrôle la taille des pas lors de la descente de gradient. Un taux trop élevé fait osciller l'apprentissage ; un taux trop faible le ralentit inutilement. Ce programme utilise un **Cosine Decay** : le taux part de 3×10^{-4} et décroît progressivement, en suivant une courbe en cosinus, jusqu'à une valeur quasi nulle (10^{-6}) en fin d'entraînement.

6.3 Callbacks

Deux mécanismes automatiques surveillent l'entraînement :

- **ModelCheckpoint** : sauvegarde automatiquement le meilleur modèle (selon la perte de validation).
- **EarlyStopping** : arrête l'entraînement si la perte de validation ne s'améliore plus après 4 époques consécutives, pour éviter le surapprentissage.

6.4 Courbes d'apprentissage

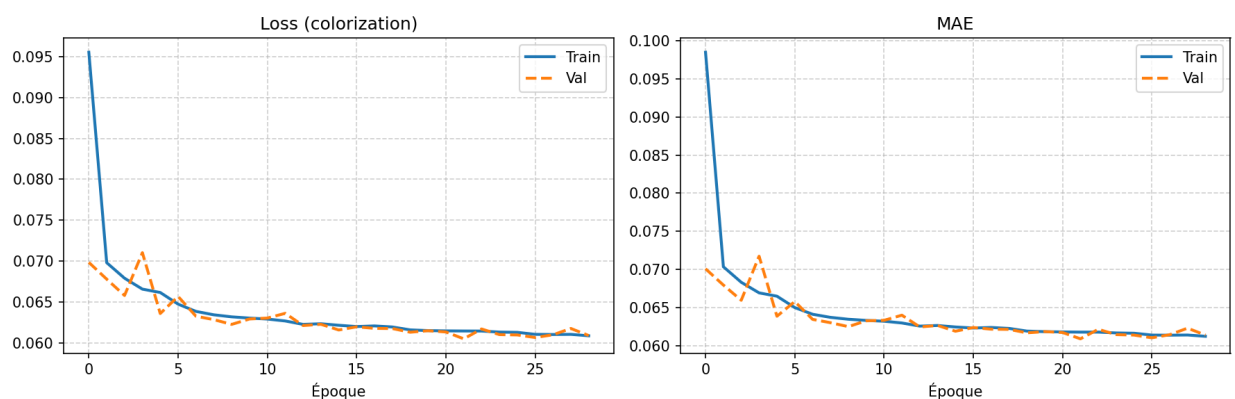


FIGURE 2 – Évolution de la perte et de la MAE au cours de l'entraînement.

7 Coloriser une image donnée

Une fois le modèle entraîné, voici comment une nouvelle image est colorisée :

1. L'image d'entrée (couleur ou niveaux de gris) est chargée et convertie en RGB.
2. Elle est redimensionnée à 128×128 et convertie en Lab.
3. Le canal L est extrait, normalisé, et passé au réseau.
4. Le réseau prédit les canaux ab_{norm} , qui sont dénormalisés ($\times 128$).
5. On reconstitue l'image Lab complète en assemblant L original + ab prédit.
6. La conversion Lab $\rightarrow$ RGB produit l'image colorisée finale.
7. Un redimensionnement vers la taille originale est effectué (via interpolation LANCZOS).

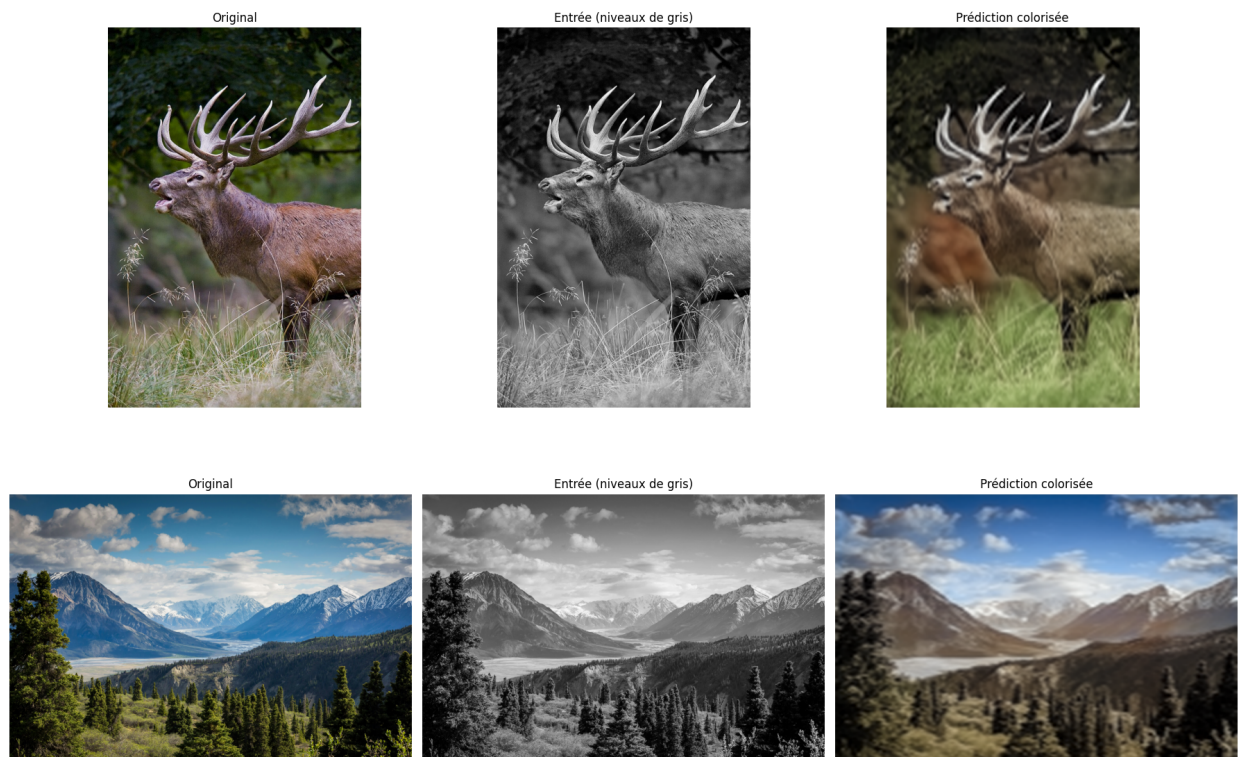


FIGURE 3 – Exemples de colorisations produites par le modèle (original, niveaux de gris, prédiction).

8 Utilisation du programme

Le programme s'utilise en ligne de commande. Les quatre opérations principales sont :

Listing 1 – Commandes principales du programme

```
1 # 1. Creer un nouveau modele
2 python colorizer.py --create mon_modele
3
4 # 2. Entrainer le modele (sur 20 epoques)
5 python colorizer.py --train mon_modele 20
6
7 # 3. Coloriser une image
8 python colorizer.py --predict mon_modele chemin/vers/image.
   jpg
9
10 # 4. Afficher les courbes d'apprentissage
11 python colorizer.py --plot mon_modele
```